

AGGREGATION PIPELINE

- MongoDB Aggregation Pipeline consist of stages and each stage transforms the document. It is a multi-stage pipeline and in each state and the documents are taken as input to produce the resultant set of documents.
- In the next stage (ID available) the resultant documents are taken as input to produce output, this process continues till the last stage.
- The basic pipeline stages are defined below:
 1. filters that will operate like queries.
 2. the document transformation that modifies the resultant document.
 3. provide pipeline provides tools for grouping and sorting documents.
- Aggregation pipeline can also be used in **sharded collection**.
- Execute Aggregation Pipeline and its operations (pipeline must contain \$match, \$group, \$sort, \$project,
- \$skip etc. students encourage to execute several queries to demonstrate various aggregation operators)

Lets Build new Dataset

- Download collection [here](#)
- Upload the new collection with name "stud".

- Aggregation `$match`

This aggregation stage behaves like a find. It will filter documents that match the query provided.

Using `$match` early in the pipeline can improve performance since it limits the number of documents the next stages must process.

- `$group` (aggregation)

The `$group` stage separates documents into groups according to a "group key". The output is one document for each unique group key.

A group key is often a field, or group of fields. The group key can also be the result of an expression. Use the `_id` field in the `$group` pipeline stage to set the group key. See below for [usage examples](#). In the `$group` stage output, the `_id` field is set to the group key for that document.

- `$sort` (aggregation)

Sorts all input documents and returns them to the pipeline in sorted order.

● Projection Operator

projection operators allow users to control queries and results. Query operators help to filter data based on specific conditions. E.g., `$eq`, `$and`, `$exists`, etc.

Projection operators in MongoDB control which fields should be shown in the results. E.g., `$project`, `$slice`, `$concat`, etc.

● MongoDB – skip() Method

In MongoDB, the `skip()` method is used to skip a specified number of documents in a query result. This allows us to control the starting point of the returned results which effectively removes the first `n` documents from the result set.

● `$limit` (aggregation)

Limits the number of documents passed to the next stage in the [pipeline](#).

● `$unwind` (aggregation)

Deconstructs an array field from the input documents to output a document for *each* element. Each output document is the input document with the value of the array field replaced by the element.

1. Find students with age greater than 23, sorted by age in descending order, and only return name and age

```
db.students6.aggregate([
  { $match: { age: { $gt: 23 } } }, // Filter students older than 23
  { $sort: { age: -1 } }, // Sort by age descending
  { $project: { _id: 0, name: 1, age: 1 } } // Project only name and
])
```

Ans. Find students with age greater than 23, sorted by age in descending order, and only return name and age.

```
db> db.students6.aggregate([
...   { $match: { age: { $gt: 23 } } }, // Filter students older than 23
...   { $sort: { age: -1 } }, // Sort by age descending
...   { $project: { _id: 0, name: 1, age: 1 } } // Project only name and age
... ])
[ { name: 'Charlie', age: 28 }, { name: 'Alice', age: 25 } ]
db>
```

2. Group students by major, calculate average age and total number of students in each major:

```
db> db.students6.aggregate([
...   { $group: { _id: "$major", averageAge: { $avg: "$age" }, totalStudents: { $sum: 1 } } }
... ])
[
  { _id: 'Mathematics', averageAge: 22, totalStudents: 1 },
  { _id: 'English', averageAge: 28, totalStudents: 1 },
  { _id: 'Computer Science', averageAge: 22.5, totalStudents: 2 },
  { _id: 'Biology', averageAge: 23, totalStudents: 1 }
]
```

3. Find students with an average score (from scores array) above 85 and skip the first document

```
db.students6.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
      averageScore: { $avg: "$scores" }
    }
  },
  { $match: { averageScore: { $gt: 85 } } },
  { $skip: 1 } // Skip the first document
])
```

Ans: Find students with an average score (from scores array) above 85 and skip the first document

```
db> db.students6.aggregate([
...   {
...     $project: {
...       _id: 0,
...       name: 1,
...       averageScore: { $avg: "$scores" }
...     }
...   },
...   { $match: { averageScore: { $gt: 85 } } }, // Filter by average score
...   { $skip: 1 } // Skip the first document
... ])
[ { name: 'David', averageScore: 93.33333333333333 } ]
db>
```